

Seminar 3

Object-Oriented Design, IV1350

Filip Kehl, fkehl@kth.se

2026-06-12

Project Members:

Filip Kehl, fkehl@kth.se

Declaration:

By submitting this assignment, it is hereby declared that all group members listed above have contributed to the solution. It is also declared that all project members fully understand all parts of the final solution and can explain it upon request.

It is furthermore declared that no part of the solution has been copied from any other source (except for resources presented in the Canvas page for the course IV1350), and that no part of the solution has been provided by someone not listed as a project member above.

1 Introduction

This report describes the implementation of the Repair Electric Bike application, which was analysed in seminar 1 and designed in seminar 2. The task in seminar 3 is to translate the design into running Java code that implements the basic flow of the use case, and to write unit tests for that code.

The requirements on the implementation are taken from the seminar 3 task description and can be summarised as follows. The program must compile and run on a standard Java installation; only the basic flow and the startup are required. The view may be replaced by a single class with hard-coded calls to the controller, but every value returned by the controller must be printed to `System.out`, and the repair order printout must also be sent to `System.out`. Exceptions must not be used, since they are the topic of seminar 4. Persistent storage and external systems are not implemented instead the integration layer keeps data in memory and the printer simply writes to `System.out`. The code must follow every guideline in chapter 6 of the textbook, in particular every public declaration must have a Javadoc comment, and the design must keep high cohesion, low coupling and good encapsulation.

The requirements on the unit tests are also taken from the seminar 3 task description. To pass with distinction the report must contain unit tests for every class in the controller, model and integration layers, except classes that only have getters and constructors that store values. Tests of methods that only produce output to `System.out` are also not required.

2 Method

2.1 Translating the design

Before any code was written, the seminar 2 design was reviewed once more against the seminar 1 requirements specification, in particular against the basic flow steps. A small number of design decisions had been left informal, for example exactly which information the diagnostic report carries and where the repair order printout is built, and these were resolved up front to keep the implementation focused.

The implementation was then carried out one system operation at a time, in the order they appear in the basic flow. The first operation, `searchCustomer`, was implemented end-to-end before any work began on the next operation. This is the same approach that is used in section 6.5 of the textbook and it made it possible to verify each piece of the design as soon as it was coded, instead of producing a large amount of unverified code.

The package layout follows the recommendation in section 6.1 of the textbook: a reverse domain prefix `se.kth.iv1350.repairbike`, followed by one sub-package per layer (startup, view, controller, model, integration). All names use the standard Java naming conventions summarized in table 6.1. Whenever a value or constant could not be expressed by a meaningful name, it was extracted to a private static final field, as described under "Anonymous Values" in section 6.4.

2.2 Differences between the seminar 2 design and the seminar 3 code

The seminar 3 task explicitly allows the design to be changed where flaws are discovered during implementation, and a few changes were made. The substantive ones are listed below, with the motivation in each case.

- Controller method names. The seminar 2 design used `findCustomer`, `createRepairOrder`, `addDiagnosticResult` and `addRepairTask`. In the code they are renamed to `searchCustomer`, `registerProblem` and `registerDiagnostic` with the two technician steps combined into a single `registerDiagnostic` call that takes the complete diagnostic report. The shorter API matches the basic-flow-only scope and removes the need for the technician to know an explicit order id, since the controller already holds the current order during the receptionist's session.
- Controller is stateful. The design returned a `RepairOrderDTO` from every operation so that the view could pass the id back on the next call. In the code the controller keeps the current repair order in a field, which is valid for the basic flow because the receptionist handles one customer at a time.
- `DiagnosticReport` carries both the description and the repair tasks. The design had a separate `addRepairTask` loop on the order. The code groups the related data in `DiagnosticReport`, which raises cohesion and lets the report compute its own total cost (`DiagnosticReport.getTotalCost`).
- The view sees the order through a `RepairOrderDTO`. Earlier versions of this implementation returned the `RepairOrder` entity to the view, which made it possible for the view to call any model method on it. To restore the design's encapsulation, `RepairOrderDTO` was reintroduced as an immutable snapshot. `RepairOrder.toDTO()` is the only way to construct one.
- `RepairOrder.accept(Printer)` does the printing itself. Earlier versions had the controller call `Printer` directly after the `accept` call. Moving the print into the model removes Controller's association to `Printer` for the print path and matches the exact improvement that the textbook applies in figure 5.42 (`Rental.printReceipt`). Controller still owns the `Printer` reference, but only in order to hand it on to `RepairOrder`. **The order does not format anything: it hands the printer a data-only `RepairOrderDTO` snapshot of itself and all formatting of the printout happens inside the printer.**
- **Printer takes a `RepairOrderDTO`, not a `RepairOrder` entity. The DTO is an immutable, data-only snapshot so the integration layer still cannot reach into or affect the model entity. The printer renders the paper printout itself, in the private method `Printer.createPrintout`: how the printed page looks is a concern of the printing device, not of the model. As a consequence, neither `RepairOrder` nor `RepairOrderDTO` contains any presentation logic. The DTO carries data only, as a DTO should.**
- `RepairOrderRegistry.saveRepairOrder` is split into `addRepairOrder` and `updateRepairOrder`. Both are explicit about their intent and mirror the signatures a real database-backed registry would need. The in-memory implementation of `updateRepairOrder` is a no-op, since the registry already holds the reference.

The alternative flows from seminar 1 (unknown phone number, repair order rejected) are not implemented, since they are explicitly outside the scope of the seminar 3 task and would benefit from the exception handling that is the topic of seminar 4.

2.3 Writing the unit tests

Unit tests were written together with the production code, not at the end. As soon as a class had its public methods, a corresponding test class was created in the same package but in the parallel `src/test/java` directory tree.

When choosing what to test, the textbook's advice in section 7.2 was followed: pure getters and pass-through constructors were skipped and output to `System.out` was never tested for its exact format. `Printer.printRepairOrder`, which renders the printout from the data in a `RepairOrderDTO`, is however tested for content in `PrinterTest`: the tests capture `System.out` and verify that the rendered printout contains the order's data, without asserting on the exact layout. Otherwise the goal was to cover every public method, every state transition of the repair order, and the boundary cases that were easy to construct, for example searching for an unknown phone number, computing the total cost of an empty diagnostic report, and verifying that `DiagnosticReport` actually takes a defensive copy of its task list.

Two situations from section 7.3 were encountered. First, the controller indirectly calls `Printer`, which writes to `System.out`. The test `ControllerTest.testAcceptRepairOrderTriggersPrintout` deals with this by temporarily replacing `System.out` with a `ByteArrayOutputStream` and inspecting the captured text. The same trick is used by `RepairOrderTest.testAcceptSendsPrintoutToPrinter` and by both tests in `PrinterTest`. These two tests only verify that something was printed and that it contains the order id and the `ACCEPTED` state, they do not test the exact format. Second, `RepairOrder` exposes its state to tests through the immutable `RepairOrderDTO` snapshot returned by `toDTO()`, this means the entity itself can keep its getters out of the public interface but the tests can still read everything they need, without breaking encapsulation.

`RepairOrderRegistry` has no getter that exposes its list of orders. Rather than adding such a getter just to enable testing, a small `numberOfRepairOrders` method was added that returns the size of the list.

3 Result

The complete source code is available in the public Git repository: <https://github.com/fikehl/iv1350-repair-electric-bike-sem3>

3.1 Program structure

The program is organised in five layers, following the MVC and Layer patterns from chapter 5 of the textbook. The startup layer contains the `Main` class, which performs the dependency wiring and starts the program. The view layer contains a single class, `View`, which simulates the user interface by making a fixed sequence of calls to the controller and printing every returned value to `System.out`. The controller layer contains the single class `Controller`, which is the only entry point from the view to the rest of the program. The model layer contains the business logic and the central entity classes: `RepairOrder`, `DiagnosticReport`, `RepairTask`, `Amount`, the enum `RepairOrderState`, and the immutable snapshot `RepairOrderDTO`. The integration layer contains the data access objects (`CustomerDTO`, `BikeDTO`), the registries (`CustomerRegistry`, `RepairOrderRegistry`, `RegistryCreator`) and the placeholder for the external printer (`Printer`).

All calls go from higher layers to lower layers, never the other way around. `View` calls `Controller`, `Controller` calls model and integration, and the model calls the integration layer only in one well-defined place: `RepairOrder.accept(Printer)` hands a data-only `RepairOrderDTO` snapshot to the printer. The printer renders the printout from that data itself so no formatted display text is ever produced inside the model. `Printer` depends only on the immutable DTO, never on the `RepairOrder` entity. This keeps coupling low and gives every package a single, well-defined responsibility.

3.2 System operations

Four system operations are exposed by the controller, exactly matching the four points in the basic flow where the actor enters data into the system: `searchCustomer`, `registerProblem`, `registerDiagnostic` and `acceptRepairOrder`. Each of these methods returns the object that the view will display, in keeping with the textbook's rule that controller methods should never be void when the view needs the result. `searchCustomer` returns a `CustomerDTO` (or null when the phone number is unknown). The three remaining operations return an immutable `RepairOrderDTO` snapshot of the current state of the order. The current repair order is held as a field on the controller. This is acceptable because the controller represents one ongoing receptionist session at a time.

The acceptance step deserves a separate comment. The controller does not call the printer directly, it calls `RepairOrder.accept(printer)`, and the order asks the printer to print itself. The controller therefore stays thin and is not coupled to the printer's interface, the same improvement is shown in figure 5.42 of the textbook.

3.3 Sample run

The complete output of running the program is shown below. The output starts with the customer lookup and ends after the receptionist has registered acceptance, which causes the printer to print the final repair order. Every value returned by the controller is also printed by the view, which is why the final repair order appears twice in the output.

The view now prints its own compact formatting of every repair order it displays (`View.formatRepairOrder`), while the boxed printout shown at the acceptance step is produced separately by the Printer from the data in a `RepairOrderDTO`.

```
#####
# Sample run of Repair Electric Bike (basic flow), seminar 3.
#
# Produced by tracing through View.sampleExecution(). Timestamps
# are taken from LocalDateTime.now() at runtime, so the exact
# dates will of course differ when the program is run again.
#
# The view prints its own formatting of every DTO returned by
# the controller. The boxed printout shown at the acceptance
# step is produced by Printer, which renders the paper copy.
#####

#####

RECEPTIONIST: Customer arrives. Searching for customer with phone 0701112233.
System returned: Customer[name=Anna Andersson, phone=0701112233, email=anna@example.com,
Bike[brand=Crescent, model=Elina E8, serialNumber=CR-E8-00417]]

#####

RECEPTIONIST: Customer describes the problem. Registering it.
System returned a new repair order:
Repair order id:      1
State:                NEWLY_CREATED
Created:              2026-06-10 14:30
Customer:             Anna Andersson, 0701112233
Bike:                 Crescent Elina E8
Problem description:  The motor cuts out after about ten minutes of use, and the front brake
squeaks loudly when applied.
Diagnostic report:    (none yet)
Estimated completion: 2026-06-17 14:30
```

```

#####

TECHNICIAN: Performing diagnostic and proposing repair tasks.
System returned the updated repair order:
Repair order id:      1
State:                READY_FOR_APPROVAL
Created:              2026-06-10 14:30
Customer:             Anna Andersson, 0701112233
Bike:                Crescent Elina E8
Problem description:  The motor cuts out after about ten minutes of use, and the front brake
squeaks loudly when applied.
Diagnostic report:    The battery management system has a faulty temperature sensor that
triggers an emergency shutdown. The front brake pads are worn down to the wear indicators.
Proposed repair tasks:
- Replace battery temperature sensor: 1200 SEK
- Replace front brake pads: 450 SEK
- Full safety inspection: 350 SEK
Total cost:          2000 SEK
Estimated completion: 2026-06-17 14:30

#####

RECEPTIONIST: Customer accepts the proposed repair tasks. Registering acceptance.
=====
REPAIR ORDER #1
=====
Date:    2026-06-10 14:30
State:   ACCEPTED
-----
Customer: Anna Andersson
Phone:    0701112233
Email:    anna@example.com
Bike:     Crescent Elina E8 (S/N CR-E8-00417)
-----
Customer's problem description:
The motor cuts out after about ten minutes of use, and the front brake squeaks loudly when
applied.
-----
Diagnostic report:
The battery management system has a faulty temperature sensor that triggers an emergency
shutdown. The front brake pads are worn down to the wear indicators.

Proposed repair tasks:
- Replace battery temperature sensor: 1200 SEK
- Replace front brake pads: 450 SEK
- Full safety inspection: 350 SEK

Total cost: 2000 SEK
-----
Estimated completion: 2026-06-17 14:30
=====

System returned the accepted repair order:
Repair order id:      1
State:                ACCEPTED
Created:              2026-06-10 14:30
Customer:             Anna Andersson, 0701112233
Bike:                Crescent Elina E8
Problem description:  The motor cuts out after about ten minutes of use, and the front brake
squeaks loudly when applied.
Diagnostic report:    The battery management system has a faulty temperature sensor that
triggers an emergency shutdown. The front brake pads are worn down to the wear indicators.
Proposed repair tasks:
- Replace battery temperature sensor: 1200 SEK
- Replace front brake pads: 450 SEK
- Full safety inspection: 350 SEK
Total cost:          2000 SEK
Estimated completion: 2026-06-17 14:30

#####

```

Figure 1: Sample run output, captured from System.out.

4 Discussion

4.1 Design principles

Cohesion. Each class has a single, focused responsibility. `RepairOrder` is the only class that holds the life cycle state of an order and it contains business logic only. All presentation has been moved out of the model: the view formats the order information it shows the user (`View.formatRepairOrder`) and the printer formats the paper printout it produces (`Printer.createPrintout`, which is split into small private helper methods such as `appendHeader` and `appendCustomerSection` so that it reads as a sequence of well-named steps). Each class formats only the output it is itself responsible for, which raises cohesion in both places. `DiagnosticReport` contains both a description and a list of repair tasks because they always belong together and it is the natural owner of the `getTotalCost` calculation, which is therefore not duplicated anywhere else in the code.

Coupling. The view depends only on the controller and on a small number of model and integration types that show up in the public interface of the controller (`CustomerDTO` and `RepairOrderDTO`). The controller depends on the integration layer through `RegistryCreator` and on the model through `RepairOrder`. It does not know how the registries store their data and it does not call the printer's hardware. The `Printer` class depends only on the immutable, data-only `RepairOrderDTO` and the small value types it carries, never on the `RepairOrder` entity so it cannot read or change the state of the model. The Layer pattern is still strictly enforced: there are no upward calls and the only downward layer crossing into integration from model is the controlled call inside `RepairOrder.accept(Printer)`, which passes a DTO and nothing else.

Encapsulation. Every field is private. Mutation of model state happens only through named methods that also enforce the corresponding state transition (`addDiagnosticReport`, `accept`). The two integration DTO classes (`CustomerDTO`, `BikeDTO`) are final and have only final fields, so they are immutable and can be shared freely between layers. `RepairOrderDTO` is also a final class with only final fields, and its constructor is package-private so that only `RepairOrder.toDTO()` can produce one. `DiagnosticReport` takes a defensive copy of the proposed task list in its constructor and exposes the list through `Collections.unmodifiableList`, so callers cannot mutate it from the outside, this is verified by `DiagnosticReportTest.testConstructorTakesDefensiveCopyOfTasks` and `testGetProposedTasksIsUnmodifiable`. `RepairOrder` itself no longer exposes its customer, problem description, creation time, diagnostic report, total cost or estimated completion time as getters. The view reads all of this through the snapshot returned by `toDTO()`, so the entity stays intact.

MVC. A defect in the previous version of this program was that `RepairOrder.createPrintout` built a formatted, human-readable printout inside the model, and that `RepairOrderDTO` carried this pre-formatted string to the view through `getPrintout`. A DTO is meant to carry data only, and formatting output intended for display is the responsibility of the view layer, so this placed presentation logic in the model and violated MVC. This has been corrected: `RepairOrderDTO` now contains only data, the view formats everything it displays itself (`View.formatRepairOrder`), and the printer renders the paper printout from the data in the DTO (`Printer.createPrintout`). The model no longer contains any presentation code, so the Model and View concerns of MVC are properly separated.

4.2 Avoided code smells

Primitive obsession is avoided by introducing Amount for money and a RepairOrderState enum for the life cycle states. The enum eliminates the kind of typo bug that the textbook warns about.

Long parameter lists are avoided. The constructor of RepairOrder takes a CustomerDTO instead of name, phone, email, brand, model and serial number separately, and Controller.registerProblem follows the same pattern.

Long methods are avoided. Printer.createPrintout is the longest method in the program and is broken into a few short, named helpers. If a reader wants to know what one section of the printout looks like, they can read just the corresponding helper.

Anonymous values are avoided. The number of days that the repair takes is named DEFAULT_REPAIR_DAYS in RepairOrder, not written as a literal 7, so the meaning is clear at the call site. The same applies to the DATE_TIME_FORMATTER constants in Printer and View, which are also private static final and therefore only allocated once.

Duplicated code is avoided. The total cost of a repair order is computed in exactly one place, DiagnosticReport.getTotalCost and RepairOrder delegates to it.

4.3 Comments

Every public class, constructor and method has a Javadoc comment that describes what it does, its parameters and its return value. The comments describe the contract, not the implementation, in keeping with section 6.3. There are no inline comments inside methods. Whenever a piece of logic could have benefited from one, a private helper with a meaningful name was extracted instead.

4.4 Unit tests

Tests exist for every non-trivial class in the controller, model and integration layers: ControllerTest, RepairOrderTest, DiagnosticReportTest, AmountTest, CustomerRegistryTest, RepairOrderRegistryTest, RegistryCreatorTest, and PrinterTest. Pure DTOs (CustomerDTO, BikeDTO, RepairTask, RepairOrderDTO) and the enum RepairOrderState are not tested because they only have getters and constructors that store values. Printer is now tested by PrinterTest, since it no longer just forwards a string: it renders the printout from the data in a RepairOrderDTO, and the tests verify that the rendered printout contains the order's data.

The tests are independent and self-evaluating: each test creates its own Controller and registries in @BeforeEach and discards them in @AfterEach, so the order in which tests are executed does not affect the result. Each test contains a small number of assertions, each with an explanatory message that pinpoints the failure if the test fails. Boundary cases that are easy to construct are covered: an empty diagnostic report should have a total cost of zero, a differently formatted phone number should not match a known customer, updateRepairOrder must not add a duplicate, two consecutive calls to nextRepairOrderId should never return the same id, and getRepairOrderRegistry must always return the same instance across repeated calls.

4.5 Limitations and possible improvements

Only the basic flow is implemented. The alternative flows from seminar 1 (unknown phone number, repair order rejection) are not implemented, since they were not required by the seminar 3 task and would benefit from the exception handling that is the topic of seminar 4.

There is no real persistent storage, no real customer registry and no real printer. The integration classes are placeholders that mimic the API of those external systems, so that they can be replaced without changing any code outside the integration package.

Phone numbers are kept as raw strings rather than as their own `PhoneNumber` type. A future version could introduce that class to encapsulate normalisation and validation, exactly as the textbook suggests for personal numbers in section 6.4.

References

Lindback, L. (2026). A First Course in Object-Oriented Development

AI usage

ChatGPT 5.5 was used to improve spelling and grammar in this report.